Facultad Regional Córdoba

ObraSmart

Plataforma Web Distribuida para la Gestión Integral de Equipos, Stock y Operaciones en Obras de Construcción

Leticia Silvana Castro Altamirano - 114179
Fernando Gabriel Ferreyra - 114320

Universidad Tecnológica Nacional

Práctica Supervisada

Ing. Mariela Fátima Mercado

02 de Febrero de 2026

Tabla de contenido

Resumen.....3

Introducción.....4

Método.....5

Límites.....5

Alcances.....5

Tecnologías Utilizadas.....9

Mejoras a Futuro.....13

Diagramas de Flujo.....14

Conclusión.....15

Referencias.....16

Figuras.....17

Apéndice.....20

Resumen

La gestión operativa de obras de construcción presenta múltiples desafíos relacionados con el control de equipos, mantenimiento, stock de repuestos, proveedores, logística y trazabilidad de recursos en tiempo real. En la práctica, muchas empresas del rubro continúan utilizando planillas de cálculo, registros manuales y sistemas aislados que generan inconsistencias, demoras operativas y falta de visibilidad global.

El objetivo de ObraSmart es diseñar e implementar una plataforma web distribuida basada en arquitectura de microservicios que permita centralizar, automatizar y optimizar la gestión integral de los recursos operativos en obras, brindando trazabilidad en tiempo real, control de stock, gestión de reparaciones, autenticación segura, y geolocalización de equipos.

INTRODUCCIÓN

La industria de la construcción opera en entornos dinámicos y distribuidos, caracterizados por múltiples frentes de obra, traslados de maquinaria y una alta dependencia de terceros. En este escenario, la administración de recursos operativos alcanza una complejidad crítica cuando la información se encuentra fragmentada entre distintas herramientas y sectores.

Históricamente, el seguimientos de equipos, repuestos y tareas de mantenimientos se ha basado en registros manuales y planillas de cálculo. Si bien ese modelo es funcional en escalas reducidas, se torna ineficiente ante el crecimiento operativo, provocando una pérdida de trazabilidad sobre el estado real de la maquinaria y disponibilidad de insumos y el historial de intervenciones técnicas realizadas.

Estas carencias derivan en costos y demoras evitables: un equipo fuera de servicio por no contar con el repuesto necesario, una reparación no registrada correctamente o una orden de compra gestionada con información desactualizada detienen la cadena productiva. Ante la ausencia de una visión global de tiempo real, la toma de decisiones estratégica se ve comprometida.

Bajo esta premisa nace ObraSmart. Aunque se concibe como un desarrollo basado en el análisis de problemáticas sectoriales recurrentes, su diseño responde a la necesidad de digitalizar y estandarizar los flujos de trabajo. ObraSmart se propone como una plataforma web integral que centraliza la información, transformando la gestión operativa en un proceso ágil, confiable y accesible desde cualquier punto de la organización.

Método

El desarrollo de la aplicación se encuadra en un proyecto de software como parte del trabajo final de la carrera. Dicho proyecto involucró la elaboración de límites y alcances que a continuación se detallan.

Límites

Desde la autenticación segura de usuarios mediante JWT, la asignación de roles y privilegios, el registro y gestión de equipos, repuestos, proveedores y reparaciones dentro del sistema.

Hasta la visualización centralizada de la información operativa en tiempo real, el control de stock actualizado, el historial de intervenciones técnicas sobre los equipos y la trazabilidad completa de los recursos a través de la interfaz web.

Alcances

ObraSmart responde al enfoque de arquitectura de microservicios independientes. cada uno es responsable de un dominio funcional específico dentro de la gestión operativa de obras.

A continuación, se detallan los alcances correspondientes a cada microservicio implementado.

Microservicio Identity Service (Autenticación y Seguridad)

- Autenticación de usuarios mediante credenciales.
- Generación y validación de tokens JWT.
- Validación de permisos y roles.
- Alta, baja y modificación , Eliminación (Baja Logica) de usuarios del sistema.
- Asignación de roles y permisos.
- Administración de perfiles de usuario.
- Consulta de usuarios registrados.

Microservicio Gestión Obrador

- Registro Alta, baja y modificación , Eliminación (Baja Logica) de obradores (lugares físicos de operación).
- Asociación de equipos a obradores.
- Consulta del estado operativo por ubicación.

- provee pares de coordenadas para renderizar el obrador en un radio

  Asignación de equipos a obradores.

Microservicio Gestión Equipos

- Registro Alta, baja y modificación , Eliminación (Baja Logica) de Equipos del sistema.

- Seguimiento del estado operativo.
- Historial de movimientos de los equipos.

Microservicio Gestión Stock

- Registro de repuestos Alta, baja y modificación , Eliminación (Baja Lógica) de Repuestos del sistema..
- Gestión de proveedores Alta, baja y modificación , Eliminación (Baja Lógica) de proveedores del sistema.
- Gestión de órdenes de compra Alta, baja y modificación , Eliminación (Baja Logica) de Órdenes de compras del sistema..
- Control de movimientos de stock (ingreso y egreso).
- Actualización automática de cantidades disponibles.

Microservicio Gestión Reparaciones

- Registro de reparaciones asociadas a equipos Alta, baja y modificación , Eliminacion de reparaciones del sistema..
- Historial de intervenciones técnicas.
- Asociación de repuestos utilizados en cada reparación.
- Seguimiento del estado de cada reparación.

Microservicio Gestión Geolocalización

- Visualización de la ubicación de equipos en el mapa.
- Representación gráfica mediante integración con Leaflet.
- Asociación de equipos con coordenadas geográficas.

Microservicio Gestión Logística

- Registro de movimientos de equipos entre obradores.
- Seguimiento de traslados.
- Historial de ubicaciones y movimientos.

Microservicio Gestión Reportes

- Generación de reportes operativos.
-
- Impresión de reportes pdf
- Consulta consolidada de información del sistema.
- Visualización de datos relevantes para la toma de decisiones.

Microservicio Gestión Asistente

- Implementación de un asistente inteligente orientado a facilitar la interacción del usuario con el sistema.
- Interpretación de consultas del usuario relacionadas con la gestión operativa.
- Integración con los microservicios para obtener y presentar información contextual.
- Bot asistente para guiar al usuario en la navegación de la aplicación.

Este módulo incorpora capacidades de inteligencia artificial dentro de la plataforma, permitiendo una interacción más intuitiva y eficiente con el sistema, Implementando un bot personalizado para consultas del entorno ,ia Local para consultas simples (no necesita internet) , ia could consultas mas complejas (utiliza internet) ,la ia que responde es automatica y es selecciona acorde a la consulta, diferenciando a ObraSmart de los sistemas tradicionales de gestión.

Microservicio Media Service

Su propósito es centralizar el manejo de archivos multimedia. A diferencia de almacenar binarios en bases de datos de otros microservicios, MediaService persiste los archivos en un volumen Docker dedicado, adoptando un modelo conceptual de *object storage* (similar a MinIO o Amazon S3).

- Almacenamiento físico de imágenes y documentos en un volumen persistente
- Recuperación de archivos mediante identificadores únicos
- Asociación de archivos a entidades del sistema (equipos, reparaciones)
- Exposición de endpoints para carga y descarga de archivos
- Abstracción del almacenamiento para el resto de los microservicios

API Gateway

- Enrutamiento centralizado de solicitudes desde el frontend.
- Gestión de seguridad y CORS.
- Comunicación unificada entre los microservicios.

Frontend Angular (frontend-obrasmart2)

- Interfaz unificada de acceso a todos los módulos.
- Visualización en tiempo real de la información.
- Experiencia de usuario centralizada.

Tecnologías Utilizadas

Para el desarrollo de **ObraSmart**, se utilizó el siguiente stack tecnológico:

Frontend

- **Angular 16**: Framework para el desarrollo de interfaces web dinámicas y modulares mediante componentes reutilizables.
- **TypeScript**: Lenguaje tipado utilizado por Angular para mejorar la mantenibilidad del código.
- **SCSS**: Para la personalización de estilos y tematización de la interfaz.
- **Bootstrap Icons / Lucide Icons**: Para la incorporación de iconografía en la interfaz.
- **Leaflet**: Biblioteca de mapas utilizada para la visualización geográfica de los equipos.
- **SweetAlert2**: Para notificaciones visuales e interacción amigable con el usuario.
- **ngx-translate**: Para la gestión de múltiples idiomas en la interfaz.

Backend

- **Java 17**: Lenguaje de programación principal.
- **Spring Boot 3.x**: Framework para el desarrollo de microservicios REST.
- **Spring Data JPA**: Para el acceso y persistencia de datos.
- **Hibernate**: ORM para el mapeo objeto-relacional.
- **Maven**: Gestión de dependencias y construcción de los proyectos.
- **Spring Cloud Gateway**: Para el enrutamiento centralizado y seguridad.
- **JWT (JSON Web Tokens)**: Para autenticación y autorización entre microservicios.
- **RestTemplate**: Para la comunicación entre microservicios.

Base de Datos

- **PostgreSQL 17**: Base de datos relacional utilizada por cada microservicio, respetando el principio de base de datos por servicio.

Contenedores y Orquestación

- **Docker**: Utilizado para ejecutar cada microservicio y el frontend dentro de contenedores independientes, garantizando portabilidad, aislamiento y consistencia entre entornos de ejecución.

- **Docker Compose**: Empleado para coordinar la ejecución conjunta de todos los contenedores que conforman el sistema, incluyendo microservicios, bases de datos y redes internas necesarias para su comunicación.

Testing

- **JUnit 5**: Framework para pruebas unitarias.

- **Mockito**: Para la simulación de dependencias en pruebas.

- **smoke test : Postman**

Control de Versiones

- **Git**: Sistema de control de versiones.

- **GitHub**: Plataforma para alojar repositorios y gestionar el código fuente.

Gestión del Proyecto

Se implementó la metodología ágil **Scrum** para la gestión de tiempos y entregas parciales del sistema funcionando.

- **Jira Software**: Para la organización de tareas, definición de sprints y seguimiento del progreso.

- Sprints de 2 semanas con entregables funcionales incrementales.

Integración de APIs Externas en ObraSmart

ObraSmart no sólo se compone de microservicios internos, sino que además integra servicios externos de terceros para enriquecer la información operativa del sistema sin necesidad de desarrollarla desde cero. Esta integración permite ampliar las capacidades funcionales manteniendo la arquitectura desacoplada y escalable.

Las APIs externas utilizadas cumplen un rol fundamental dentro del sistema, principalmente en el módulo de geolocalización, logística y de asistente.

- **OpenStreetMap + Leaflet — Renderización:** Para la representación visual de equipos y obradores en el mapa, el frontend Angular utiliza la biblioteca Leaflet, la cual consume cartografía proveniente de OpenStreetMap

- **Nominatim — Geocodificación de Direcciones:** Para transformar direcciones físicas de obradores en coordenadas geográficas (latitud y longitud), el sistema utiliza el **servicio de geocodificación Nominatim de OpenStreetMap.**

- **OpenRouteService — Cálculo de Distancias y Rutas (Logística)**:En el módulo de logística, ObraSmart necesita calcular distancias reales entre dos puntos geográficos (por ejemplo, traslado de un equipo desde un obrador hacia otro).

- **Ollama — Ejecución Local de Modelos de Lenguaje:** permite ejecutar modelos de lenguaje (LLMs) de manera local en la máquina donde se encuentra desplegado el sistema, exponiendo una API REST que puede ser consumida por los microservicios de ObraSmart.

- **Mistral (IA en la nube):** soporte de procesamiento de lenguaje natural para consultas complejas del asistente.

- **I18N :** Implementamos internacionalización (i18n) en el frontend mediante ngx-translate, que permite carga dinámica de traducciones desde archivos JSON y cambio de idioma en tiempo real sin recargar la aplicación. Persistencia de idioma (localStorage).Preparado para más idiomas. Separación por dominios (users., equipment., etc.

Dificultades técnicas encontradas durante el desarrollo

Durante el desarrollo del producto, las mayores dificultades técnicas estuvieron relacionadas con la implementación y correcta configuración de la arquitectura de microservicios desplegada en contenedores Docker.

La integración de múltiples microservicios, cada uno con su propia base de datos PostgreSQL, requirió un profundo trabajo de configuración de redes internas en Docker, definición de variables de entorno, mapeo de puertos y resolución de problemas de conectividad entre contenedores. En particular, la comunicación entre los servicios a través del API Gateway y la correcta propagación del token JWT en cada solicitud representaron uno de los desafíos más importantes del proyecto.

Asimismo, se presentaron dificultades en la configuración de políticas de CORS y enrutamiento desde el frontend Angular hacia el Gateway, lo que demandó un análisis detallado del flujo de las peticiones HTTP dentro del ecosistema distribuido.

Otra complejidad relevante estuvo vinculada al principio de “base de datos por microservicio”. Esto implicó diseñar entidades y relaciones que, en un modelo monolítico tradicional, se resolverán mediante claves foráneas directas, pero que en esta arquitectura debieron resolverse mediante comunicación entre servicios y consistencia lógica de datos.

Un desafío técnico adicional surgió en la integración de mapas y funcionalidades de geolocalización dentro del frontend. La implementación del módulo de geolocalización mediante la biblioteca Leaflet implicó trabajar con coordenadas geográficas, representación visual de ubicaciones y asociación dinámica de los equipos a posiciones reales en el mapa. Esta integración requirió coordinar datos provenientes del backend con la representación gráfica en el frontend, resolviendo problemas vinculados a la actualización en tiempo real de ubicaciones, manejo de eventos del mapa y adaptación de la interfaz a distintos niveles de zoom y visualización.

Si bien estos obstáculos fueron resueltos con éxito, implican un tiempo de investigación y prueba superior al inicialmente previsto. No obstante, este proceso permitió consolidar una arquitectura robusta, alineada con prácticas modernas de desarrollo de sistemas distribuidos.

Mejoras a Futuro

- **Desarrollo de una aplicación móvil**, que permita a los responsables de obra consultar el estado de los equipos y registrar eventos directamente desde el terreno.

- **Ampliación del módulo de reportes** incorporando paneles de Business Intelligence (BI) para el análisis histórico de datos operativos.

- **Extensión del módulo de geolocalización**, incorporando rutas de traslado y seguimiento histórico de movimientos en el mapa.

- **Incorporación de geolocalización en tiempo real mediante sensores IoT**, permitiendo conocer la ubicación exacta de los equipos durante los traslados entre obradores, con actualización continua de posición.

- **Ampliación del sistema hacia la gestión de flota de transporte de empleados de la empresa** (camionetas, utilitarios y vehículos de apoyo), integrando sensores que permitan medir:

    - Consumo y carga de combustible.

    - Excesos de velocidad.

    - Horarios de encendido y apagado de motores.

    - Rutas realizadas por cada vehículo.

- **Evolución del módulo de Asistente Inteligente**, permitiendo consultas más complejas sobre el estado del sistema mediante procesamiento de lenguaje natural.

Estas mejoras pueden incorporarse progresivamente sin necesidad de modificar la arquitectura actual, demostrando la flexibilidad y escalabilidad del diseño adoptado en ObraSmart, y permitiendo su evolución hacia un sistema integral de monitoreo operativo y telemetría para empresas constructoras.

Diagramas de Flujo

Se presentan a continuación una serie de diagramas que representan los aspectos estructurales más relevantes del funcionamiento de ObraSmart. Estos diagramas permiten visualizar de manera clara cómo está construida la plataforma, cómo se comunican sus componentes y cómo se encuentra desplegada en un entorno dockerizado.

Los diagramas incluidos corresponden a:

- Proceso de autenticación de usuarios mediante JWT, evidenciando el rol del Gateway en la validación de seguridad antes de permitir el acceso a los servicios.

- Estructura de despliegue mediante Docker Compose, representando la red interna de contenedores y la distribución de los servicios que componen la plataforma.

Estos diagramas se referencian en el apartado **Figuras** del presente documento y permiten comprender el funcionamiento integral de ObraSmart desde el punto de vista arquitectónico y de despliegue.

Conclusión

La gestión de recursos en obras de construcción representa un desafío significativo cuando la información se encuentra dispersa en múltiples herramientas y procesos manuales, generando ineficiencias, demoras y dificultades en la toma de decisiones.

A través del desarrollo de ObraSmart, se demuestra que es posible abordar esta problemática mediante una solución tecnológica basada en microservicios y una interfaz web unificada, capaz de concentrar los datos relevantes y ofrecer una visión clara del estado de equipos, repuestos y reparaciones.

El sistema no solo mejora la organización interna, sino que establece una base sólida para futuras ampliaciones vinculadas a sensores, monitoreo de flota y asistencia inteligente, acompañando la evolución tecnológica del sector.

De esta manera, ObraSmart se presenta como una herramienta eficaz para empresas constructoras, mostrando cómo el uso de tecnologías actuales permite ordenar, unificar y hacer visible la información necesaria para la gestión de obras.

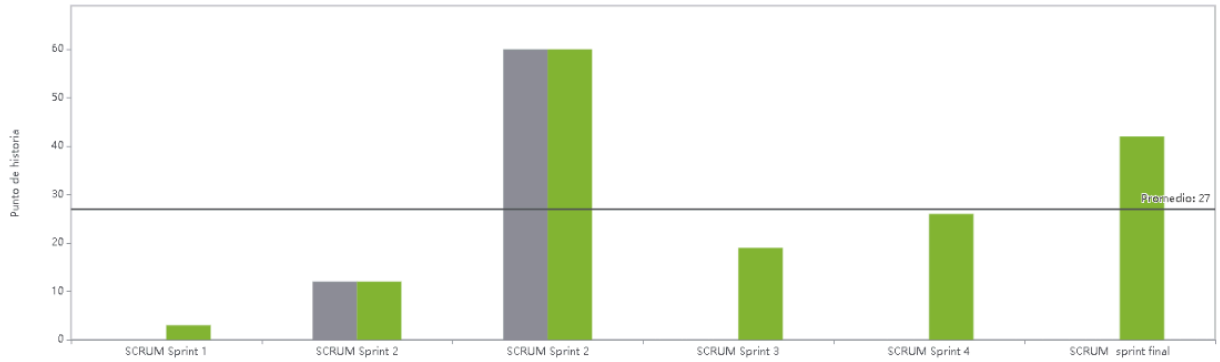
Referencias

- Docker Inc. (2024). *Docker Documentation*. Recuperado de: https://docs.docker.com/
- PostgreSQL Global Development Group. (2024). *PostgreSQL Documentation*. Recuperado de: https://www.postgresql.org/docs/
- Spring. (2024). *Spring Boot Reference Documentation*. Recuperado de: https://docs.spring.io/spring-boot/docs/current/reference/html/
- Spring. (2024). *Spring Cloud Gateway Documentation*. Recuperado de: https://docs.spring.io/spring-cloud-gateway/docs/current/reference/html/
- Oracle. (2024). *Java Platform, Standard Edition Documentation*. Recuperado de: https://docs.oracle.com/en/java/
- Angular Team. (2024). *Angular Documentation*. Recuperado de: https://angular.io/docs
- Leaflet. (2024). *Leaflet Documentation*. Recuperado de: https://leafletjs.com/
- OWASP Foundation. (2023). *JSON Web Token (JWT) Security Guidelines*. Recuperado de: https://owasp.org/
- Atlassian. (2024). *Jira Software Documentation*. Recuperado de: https://support.atlassian.com/jira-software-cloud/
- GitHub. (2024). *GitHub Documentation*. Recuperado de: https://docs.github.com/

Figuras

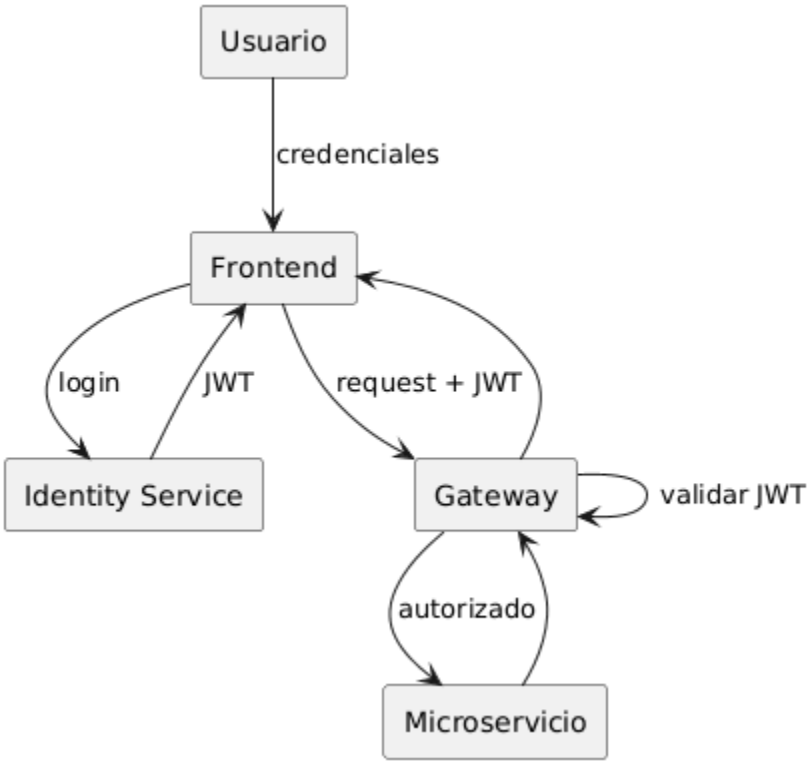
Figura 1

Roadmap del proyecto en Jira, dividido en sprints de dos semanas



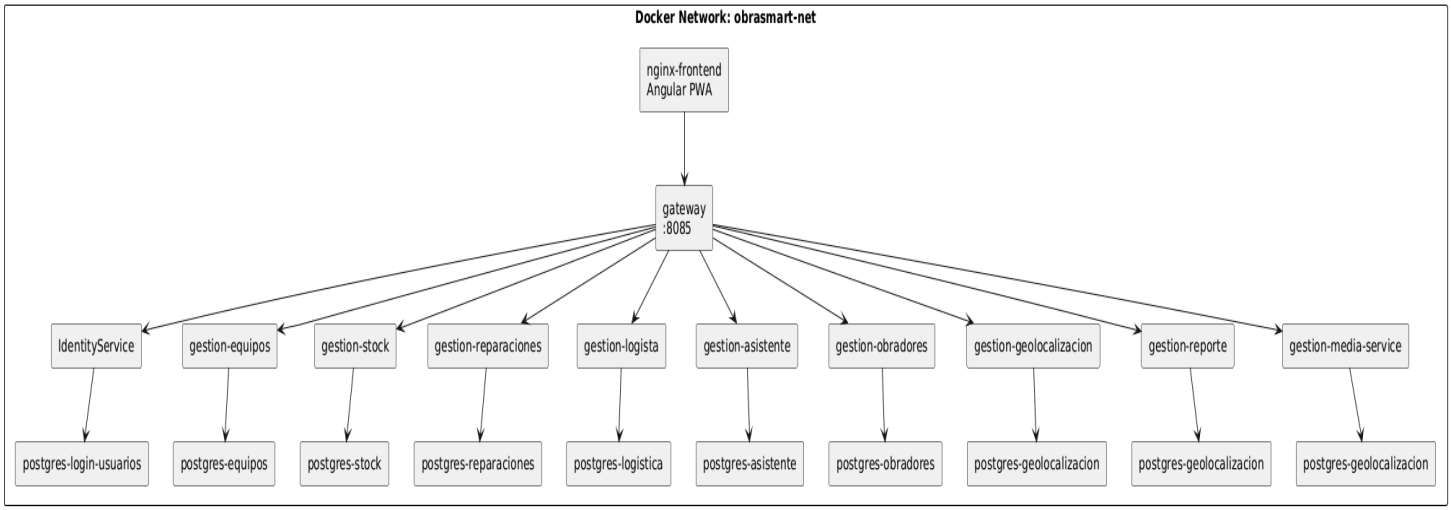
Sprint	Confirmado	Completado
SCRUM Sprint 1	0	3
SCRUM Sprint 2	12	12
SCRUM Sprint 2	60	60
SCRUM Sprint 3	0	19
SCRUM Sprint 4	0	26
SCRUM sprint final	0	42

Figura 2



Flujo de autenticación mediante JWT. Proceso de login, generación del token y validación en el Gateway antes de acceder a los servicios.

Figura 3



Estructura de contenedores definida en Docker Compose para la plataforma ObraSmart, mostrando la red interna compartida, el frontend servido por Nginx, el API Gateway y cada microservicio con su base de datos PostgreSQL independiente.

Apéndice

Enlace al proyecto en Jira:

https://tecnicatura-team-jr0pwqa9.atlassian.net/jira/software/projects/SCRUM/boards/1/timeline

Enlace al repositorio de la aplicación:

https://github.com/Trabajo-Final-Integrador/ObraSmart.git